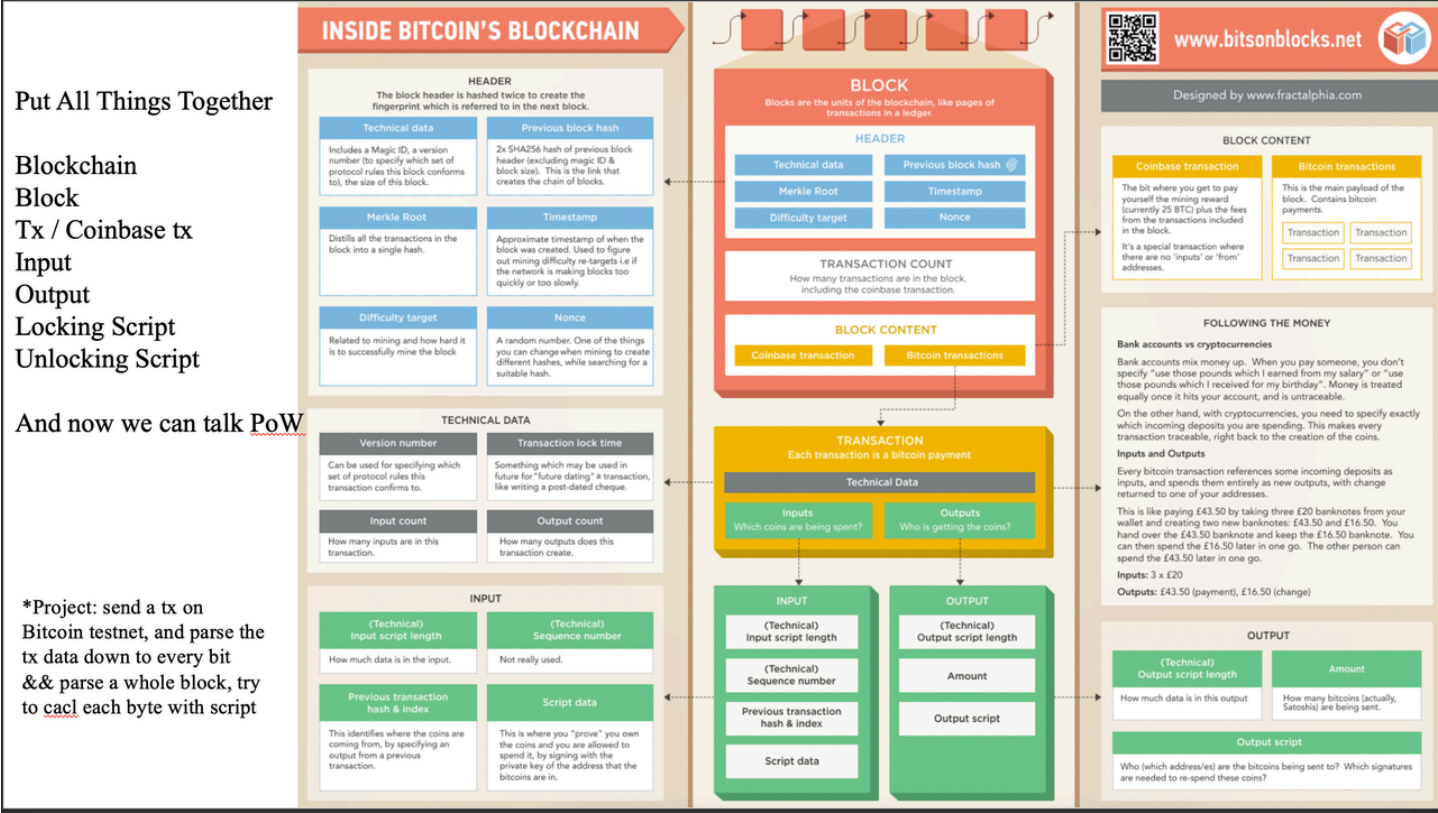


网络空间安全创新创业实践作业一、二

作业一：Bitcoin 区块链交易与区块数据解析实验



一、实验背景

随着区块链技术的发展，比特币作为最早实现去中心化数字货币系统的区块链，其底层数据结构已经成为研究区块链技术的重要基础。

比特币系统通过区块组织交易数据，并利用密码学哈希函数、Merkle Tree、工作量证明（Proof of Work, PoW）以及 UTXO（Unspent Transaction Output）模型保证系统安全性。

一个 Bitcoin 区块主要由Block Header（区块头）和Block Content（区块主体）两部分组成。其中区块头结果如下：

代码块

- 1 Block Header
- 2 |
- 3 |—— Version (版本号)
- 4 |—— Previous Block Hash (前一区块哈希)
- 5 |—— Merkle Root (Merkle 根)
- 6 |—— Timestamp (时间戳)
- 7 |—— Difficulty Target (难度目标)

这些字段共同参与 PoW 挖矿过程。

区块主体主要包含Coinbase Transaction（矿工奖励交易）和Ordinary Transactions（普通交易）部分。同时

每一笔交易transaction结构如下：

代码块

```
1 Transaction
2 |
3 |—— Version
4 |—— Input
5 |—— Output
6 |—— Locktime
```

其中Input用于引用过去交易产生的UTXO，并通过Unlocking Script提供花费证明；Output则通过Locking Script指定未来花费条件。

因此，通过对Bitcoin原始交易和区块数据进行解析，可以深入理解区块链数据结构、交易验证机制以及PoW共识过程。

二、实验目的

2.1 理解 Bitcoin 区块链数据结构

掌握Bitcoin Block、Transaction、Input、Output等核心数据结构之间的关系。理解：

代码块

```
1 Blockchain
2 |
3 Block
4 |
5 Transaction
6 |
7 Input
8 |
9 Output
```

的数据组织方式。

2.2 掌握 Bitcoin 交易解析方法

通过解析 Bitcoin Testnet 原始交易数据，获取交易版本号、Previous Transaction Hash、ScriptSig、输出数量、ScriptPubKey 以及 Locktime。

2.3 理解 Bitcoin Script 机制

通过对 Unlocking Script及 Locking Script 进行字节级解析，理解 Bitcoin 如何通过脚本实现身份验证、UTXO 解锁与地址绑定。

2.4 理解区块头结构和 PoW 原理

解析 Block Header 信息，进一步理解：区块链不可篡改性、哈希链接结构、挖矿过程。

三、实验环境

软件	版本
操作系统	Windows
Python	3.11
开发环境	PyCharm / VS Code
网络接口	Blockstream Testnet API

Python依赖库：主要依赖 requests 库，用于访问 Bitcoin Testnet API。

四、实验原理

4.1 Bitcoin Transaction 数据结构

Bitcoin 交易结构如下：

代码块

```
1 Transaction
2 |
3 |— Version
4 |— Input Count
5 |— Inputs
6 |   |— Previous Transaction Hash
7 |   |— Output Index
8 |   |— Unlocking Script (ScriptSig)
9 |   |— Sequence
10 |— Output Count
11 |— Outputs
12 |   |— Value
13 |   |— Locking Script (ScriptPubKey)
14 |— Locktime
```

其中各字段作用如下：

- Version：交易版本号；
- Input Counter：交易输入数量；
- Previous Transaction Hash：引用上一笔交易；
- Output Index：引用上一笔交易中的第几个输出；
- ScriptSig：解锁脚本，用于证明拥有该 UTXO 的花费权限；
- Sequence：序列号，可用于时间锁等高级功能；
- Output Counter：交易输出数量；
- Value：输出金额，单位为 Satoshi；
- ScriptPubKey：锁定脚本，用于规定未来花费该输出需要满足的条件；
- Locktime：交易锁定时间；

4.2 Bitcoin Script 数据解析原理

Bitcoin 中的交易脚本（Script）采用栈式脚本语言表示，每个脚本均由若干字节（Byte）组成，每个字节对应一个操作码（Opcode）或数据。为了进一步观察 Bitcoin 底层数据表示形式，本实验将脚本中的每一个字节转换为对应的二进制表示。例如：

代码块

```
1  Byte : 0x47
2  ↓
3  Binary : 01000111
```

通过这种方式，可以更加直观地观察 Bitcoin 原始交易数据的底层编码格式，并为后续分析 Bitcoin Script 的执行过程提供基础。

五、实验设计

5.1 总体流程

代码块

```
1  生成测试网钱包 (PrivateKeyTestnet)
2      |
3      ▼
4  从水龙头获取测试币
5      |
6      ▼
7  获取地址余额
```

```
8      |
9      ▼
10     输入接收方地址 + 发送金额
11     |
12     ▼
13     构造交易 (create_transaction)
14     |
15     ▼
16     广播交易 (POST /api/tx)
17     |
18     ▼
19     自动获取 TxID
20     |
21     ▼
22     调用 Blockstream API 获取 Raw Transaction
23     |
24     ▼
25     解析 Transaction (parse_tx)
26     |
27     ▼
28     解析 Input / Output / Script (含字节级展示)
29     |
30     ▼
31     输入 Block Hash (或直接按 Enter 自动获取)
32     |
33     ▼
34     调用 Blockstream API 获取区块数据
35     |
36     ▼
37     解析 Block Header (parse_block)
38     |
39     ▼
40     输出解析结果
```

5.2 程序模块设计

实验程序由三个模块组成：

(1) main.py

要负责整个实验流程控制，包括：

- 使用 `bit` 库生成测试网钱包并获取余额；
- 构造并广播交易到 Bitcoin Testnet；
- 自动获取交易ID并调用 `parse_tx()` 完成交易解析；

- 输入区块哈希并调用 `parse_block()` 完成区块解析。

代码块

```
1  import requests
2  from tx_parser import parse_tx
3  from block_parser import parse_block
4  from bit import PrivateKeyTestnet
5
6  print("=" * 50)
7  print("Bitcoin Testnet 完整工具")
8  print("=" * 50)
9  # ===== 1. 初始化钱包 =====
10 print("\n【1】初始化钱包")
11 WIF = "cUfrQQqtPc5GKpXgUmZP2YytA5kvoJdbV7jhxSkS6pHmjLgsB9vQ"
12 key = PrivateKeyTestnet(WIF)
13 print(f"当前钱包地址: {key.address}")
14 # ===== 2. 获取余额 =====
15 balance = int(key.get_balance())
16 print(f"当前余额: {balance} Satoshi")
17 if balance == 0:
18     print("\n 余额为0! 请先去水龙头领取测试币:")
19     print("    https://testnet-faucet.mempool.co/")
20     print(f"    领取地址: {key.address}")
21     exit()
22 # ===== 3. 发送交易 =====
23 print("\n【2】发送交易")
24 receiver = input("请输入接收方测试网地址: ")
25 amount = int(input("请输入发送金额(Satoshi): "))
26
27 if amount > balance:
28     print(f"余额不足! 当前余额 {balance}, 需要 {amount}")
29     exit()
30 print("正在创建交易...")
31 raw_tx = key.create_transaction([(receiver, amount, 'satoshi')])
32 print("正在广播交易...")
33 try:
34     response = requests.post(
35         "https://blockstream.info/testnet/api/tx",
36         data=raw_tx,
37         headers={"Content-Type": "text/plain"}
38     )
39     if response.status_code == 200:
40         tx_id = response.text.strip()
41         print("交易发送成功!")
42         print(f"TxID: {tx_id}")
```

```

43         print(f"查看: https://blockstream.info/testnet/tx/{tx_id}")
44     else:
45         print(f"广播失败: {response.status_code}")
46         print(response.text)
47         exit()
48 except Exception as e:
49     print(f"网络错误: {e}")
50     exit()
51 # ===== 4. 解析刚发送的交易 =====
52 print("\n【3】解析交易数据")
53 print("-" * 50)
54 raw_tx_data = requests.get(
55     f"https://blockstream.info/testnet/api/tx/{tx_id}/hex"
56 ).text
57 parse_tx(raw_tx_data)
58 # ===== 5. 解析区块 =====
59 print("\n【4】解析区块数据")
60 print("-" * 50)
61 block_hash = input("请输入区块哈希: ")
62 if block_hash == "":
63     block_hash = requests.get(
64         "https://blockstream.info/testnet/api/blocks/tip/hash"
65     ).text.strip()
66     print(f"自动获取最新区块: {block_hash}")
67 parse_block(block_hash)
68 print("\n" + "=" * 50)
69 print("全部完成!")
70

```

(2) tx_parser.py

负责交易数据解析，主要包括：

- VarInt 解析；
- Transaction Version 解析；
- Input 解析；
- Output 解析；
- ScriptSig、ScriptPubKey 提取；
- Byte/Bit 展示。

主要函数：

- read_varint()
- parse_tx()

- byte_bit_show()

代码块

```
1  import struct
2  def read_varint(data,pos):
3      x=data[pos]
4
5      if x < 0xfd:
6          return x,pos+1
7
8      elif x==0xfd:
9
10         return (
11             int.from_bytes(
12                 data[pos+1:pos+3],
13                 "little"
14             ),
15             pos+3
16         )
17 def byte_bit_show(data):
18
19     print("\n===== BYTE / BIT =====")
20     for i,b in enumerate(data):
21         print(
22             f"byte {i}: {hex(b)}  {b:08b}"
23         )
24 def parse_tx(tx_hex):
25     print("===== TX =====")
26     data=bytes.fromhex(tx_hex)
27     pos=0
28     # version
29     version=data[pos:pos+4]
30     pos+=4
31     print(
32         "Version:",
33         version.hex()
34     )
35     # input
36     vin,pos=read_varint(
37         data,
38         pos
39     )
40     print(
41         "Input:",
42         vin
```



```

43     )
44     for i in range(vin):
45         print("\nInput",i)
46         txid=data[pos:pos+32]
47         pos+=32
48         print(
49             "Previous hash:",
50             txid[::-1].hex()
51         )
52         index=data[pos:pos+4]
53         pos+=4
54         print(
55             "Index:",
56             int.from_bytes(
57                 index,
58                 "little"
59             )
60         )
61         script_len,pos=read_varint(
62             data,
63             pos
64         )
65         script=data[
66             pos:
67             pos+script_len
68         ]
69         pos+=script_len
70         print(
71             "Unlocking Script:",
72             script.hex()
73         )
74         byte_bit_show(script)
75         pos+=4    #sequence
76     # output
77     vout,pos=read_varint(
78         data,
79         pos
80     )
81     print(
82         "\nOutput:",
83         vout
84     )
85     for i in range(vout):
86         print("\nOutput",i)
87         value=data[pos:pos+8]
88         pos+=8
89         print(

```

```

90         "Value:",
91         int.from_bytes(
92             value,
93             "little"
94         )
95     )
96     sl,pos=read_varint(
97         data,
98         pos
99     )
100     script=data[
101         pos:
102         pos+sl
103     ]
104     pos+=sl
105     print(
106         "Locking Script:",
107         script.hex()
108     )
109     byte_bit_show(script)
110     lock=data[pos:pos+4]
111     print(
112         "Locktime:",
113         lock.hex()
114     )

```

(3) block_parser.py

负责区块信息解析。

主要包括：

- Block Height;
- Version;
- Previous Block Hash;
- Merkle Root;
- Timestamp;
- Difficulty;
- Nonce。

主要函数：

- parse_block()

```

1  import requests
2  API="https://blockstream.info/testnet/api"
3  def parse_block(block_hash):
4      url=f"{API}/block/{block_hash}"
5      block=requests.get(url).json()
6      print(
7          """
8          ===== BLOCK =====
9          """
10     )
11     print(
12         "Height:",
13         block["height"]
14     )
15     print(
16         "Version:",
17         block["version"]
18     )
19     print(
20         "Previous hash:",
21         block["previousblockhash"]
22     )
23     print(
24         "Merkle Root:",
25         block["merkle_root"]
26     )
27     print(
28         "Timestamp:",
29         block["timestamp"]
30     )
31     print(
32         "Nonce:",
33         block["nonce"]
34     )
35
36     print(
37         "Difficulty:",
38         block["difficulty"]
39     )

```

六、实验过程与结果

6.1 获取测试交易

本实验使用 Bitcoin Testnet（测试网）进行交易发送和解析。

1.首先通过 `bit` 库生成测试网钱包：

测试网地址: `mmVL6H7HU1TWyf6RH735dYxpjdAMEoSyAd`

2.从水龙头 (<https://coinfaucet.eu/en/btc-testnet/>) 领取测试币后，使用 Python 程序构造并广播交易。

3.交易信息：

发送方: `mxHfYitTr15xLoRGRg3d2caU5MDFcALZut`

接收方: `mmVL6H7HU1TWyf6RH735dYxpjdAMEoSyAd`

发送金额: `70110 Satoshi (0.00070110 tBTC)`

4.广播成功，程序自动从 API 响应中获取交易 ID：

`51caae8e8a0250999dfefa5cd199161eba9d651fefbb6e5e36d710264049ac39`

5.链上确认信息：

交易大小: `225 B`

手续费: `0.00000904 tBTC (904 Satoshi)`

确认数: `2 confirmations`

【1】初始化钱包

当前钱包地址: `mxHfYitTr15xLoRGRg3d2caU5MDFcALZut`

当前余额: `270110 Satoshi`

【2】发送交易

请输入接收方测试网地址: `mmVL6H7HU1TWyf6RH735dYxpjdAMEoSyAd`

请输入发送金额(Satoshi): `70110`

正在创建交易...

正在广播交易...

☒ 交易发送成功!

TxID: `51caae8e8a0250999dfefa5cd199161eba9d651fefbb6e5e36d710264049ac39`

查看: <https://blockstream.info/testnet/tx/51caae8e8a0250999dfefa5cd199161eba9d651fefbb6e5e36d710264049ac39>

STATUS	2 Confirmations
INCLUDED IN BLOCK	00000000e62f7e498fac2c3e36cf69bcc7f3dbe58113db7db1093ddba74bfc80
BLOCK HEIGHT	5075730
BLOCK TIMESTAMP	2026-07-20 23:49:40 GMT +8
TRANSACTION FEES	0.00000904 tBTC (4.02 sat/vB)
SIZE	225 B
VIRTUAL SIZE	225 vB
WEIGHT UNITS	900 WU
VERSION	1
LOCK TIME	0
SEGWIT FEE SAVINGS	This transaction could save 35% on fees by upgrading to native SegWit-Bech32 or 26% by upgrading to SegWit-P2SH
PRIVACY ANALYSIS	Address reuse

51caae8e8a0250999dfefa5cd199161eba9d651fefbb6e5e36d710264049ac39	Details
#0 4ae4af4c9287966b908c152260cc9d2817bbb4575fc1 0.00270110 tBTC 9796d87d060b76b6b850:1 #0 mmVL6H7HU1TWyf6RH735dYxpjdAMEoSAd 0.00070110 tBTC #1 mxHfYitTr15xLoRGRg3d2caU5MDFcALZut 0.00199096 tBTC	2 CONFIRMATIONS 0.00269206 tBTC

6.2 Transaction解析结果

运行程序后，`parse_tx()` 函数输出以下解析结果：

1.交易基本信息：

字段	十六进制	十进制
Version	01000000	1
Input 数量	01	1
Output 数量	02	2
Locktime	00000000	0

2.Input 0 解析：

字段	值
Previous Transaction Hash	4ae4af4c9287966b908c152260cc9d2817bbb4575fc19796d87d060b76b6b850
Output Index	1
输入金额	270110 Satoshi (0.00270110 tBTC)

2.Unlocking Script：

```
47304402205ab06d314579d486db7c85df699f7d7c1bf4731e676c5dd1fd27e1c5f93d1b6d022072ae6ab1e6203b53339e82b72c8367a10b0375b9ef6df074626dc6748e7d6872012102754283a3607c9773af09ed62b36e776a83e1dac8d6643e5b1f7e1ae4786e882a
```

3.Output 0 解析（转账）：

字段	值
Value	70110 Satoshi (0.00070110 tBTC)
接收地址	mmVL6H7HU1TWyf6RH735dYxpjdAMEoSAd
Locking Script	76a91441821e33c05a8de315e7c08d463dbc5f30b0036588ac

4.Output 1 解析（找零）：

字段	值
Value	199096 Satoshi (0.00199096 tBTC)
接收地址	mxHfYitTr15xLoRGRg3d2caU5MDFcALZut（找零返回发送方）
Locking Script	76a914b7f6c0454a0b03197868db4c60520041f2ea700988ac

5.手续费验证：

手续费 = 输入金额 - 输出金额 = 270110 - 70110 - 199096 = 904 Satoshi

```
【3】解析交易数据
===== TX =====
Version: 01900000
Input: 1

Input 0
Previous hash: 4ae4f4c9287966b908c152260cc9d2817bbb4575fc19796d874060b76b6b850
Index: 1
Unlocking Script: 4730402205ab06d3145794486db7c85df699f7d7c1bf4731e676c5dd1f427
e1c5f93d1b64022072ae6ab1e6203b53339e82672c8367a10a0373b9ef64f074626dc6748e7d6872
012102734283a3607c9773af09ed62936e776a83e1dac8d6643e3b1f7e1ae4786e882a

===== BYTE / BIT =====
byte 0: 0x47 01000111
byte 1: 0x30 00100000
byte 2: 0x44 01000100
byte 3: 0x2 00000010
byte 4: 0x20 00100000
byte 5: 0x5a 01011010
byte 6: 0xb0 10110000
byte 7: 0x6d 0101101
byte 8: 0x31 00110001
byte 9: 0x45 01000101
byte 10: 0x79 01111001
byte 11: 0xd4 11010100
byte 12: 0x86 10000110
byte 13: 0x0b 10011011
byte 14: 0x7c 01111100
byte 15: 0x85 10000101
byte 16: 0x6f 10011111
byte 17: 0x69 01101001
byte 18: 0x9f 10011111
byte 19: 0x7d 01111101
byte 20: 0x7c 01111100
byte 21: 0x1b 00011011
byte 22: 0x4 11101000
byte 23: 0x73 01110011
byte 24: 0x1e 00011110
byte 25: 0x67 01100111

byte 26: 0x6c 01101100
byte 27: 0x3d 01011101
byte 28: 0xd1 11010001
byte 29: 0xf4 11111101
byte 30: 0x27 00100111
byte 31: 0xe1 11100001
byte 32: 0x3 11000101
byte 33: 0xf9 11111001
byte 34: 0x3d 00111101
byte 35: 0x1b 00011011
byte 36: 0x6d 01101101
byte 37: 0x2 00000010
byte 38: 0x20 00100000
byte 39: 0x72 01100101
byte 40: 0xae 10101110
byte 41: 0x6a 01101010
byte 42: 0xb1 10110001
byte 43: 0xe6 11100110
byte 44: 0x20 00100000
byte 45: 0x3b 00111011
byte 46: 0x53 01010011
byte 47: 0x33 00110011
byte 48: 0x9e 10011110
byte 49: 0x82 10000010
byte 50: 0xb7 10110111
byte 51: 0x2c 00101100
byte 52: 0x83 10000011
byte 53: 0x67 01100111
byte 54: 0xa1 10100001
byte 55: 0xb 00001011
byte 56: 0x3 00000011
byte 57: 0x75 01110101
byte 58: 0xb9 10110011
byte 59: 0xef 11011111
byte 60: 0x6d 01101101
byte 61: 0xf0 11110000
byte 62: 0x74 01110100
byte 63: 0x62 01100010
byte 64: 0x6d 01101101
byte 65: 0x6 11000110

byte 66: 0x74 01110100
byte 67: 0x8e 10001110
byte 68: 0x7d 01111101
byte 69: 0x68 01101000
byte 70: 0x72 01110010
byte 71: 0x1 00000001
byte 72: 0x21 00100001
byte 73: 0x2 00000010
byte 74: 0x75 01110101
byte 75: 0x42 01000010
byte 76: 0x83 10000011
byte 77: 0xa3 10100011
byte 78: 0x6d 01100000
byte 79: 0x7c 01111100
byte 80: 0x97 10010111
byte 81: 0x73 01110011
byte 82: 0xaf 10101111
byte 83: 0x9 00001001
byte 84: 0x6d 11011101
byte 85: 0x62 01100010
byte 86: 0xb3 10110011
byte 87: 0x6e 01011110
byte 88: 0x77 01110111
byte 89: 0x6a 01101010
byte 90: 0x83 10000011
byte 91: 0xe1 11100001
byte 92: 0xda 11011010
byte 93: 0xc8 11001000
byte 94: 0x6e 11010110
byte 95: 0x64 01100100
byte 96: 0x3e 00111110
byte 97: 0x5b 01011011
byte 98: 0x1f 00011111
byte 99: 0x7e 01111110
byte 100: 0x1a 00011010
byte 101: 0xe4 11100100
byte 102: 0x78 01110000
byte 103: 0x6e 01101110
byte 104: 0x88 10001000
byte 105: 0x2a 00101010

byte 105: 0x2a 00101010

Output: 2

Output 0
Value: 70110
Locking Script: 76a91441821e33c05a8de315e7c08d463dbc5f30b0036588ac

===== BYTE / BIT =====
byte 0: 0x76 01110110
byte 1: 0xa9 10101001
byte 2: 0x14 00010100
byte 3: 0x41 01000001
byte 4: 0x82 10000010
byte 5: 0x1e 00011110
byte 6: 0x33 00110011
byte 7: 0xc0 11000000
byte 8: 0x5a 01011010
byte 9: 0x8d 10001101
byte 10: 0xe3 11100011
byte 11: 0x15 00010101
byte 12: 0xe7 11100111
byte 13: 0xc0 11000000
byte 14: 0x8d 10001101
byte 15: 0x46 01000110
byte 16: 0x3d 00111101
byte 17: 0xbc 10111100
byte 18: 0x5f 01011111
byte 19: 0x30 00110000
byte 20: 0xb0 10110000
byte 21: 0x3 00000011
byte 22: 0x65 01100101
byte 23: 0x88 10001000
byte 24: 0xac 10101100

Output 1
Value: 199096
Locking Script: 76a914b7f6c0454a0b03197868db4c60520041f2ea700988ac

===== BYTE / BIT =====
byte 0: 0x76 01110110
byte 1: 0xa9 10101001
byte 2: 0x14 00010100
byte 3: 0xb7 10110111
byte 4: 0xf6 11110110
byte 5: 0xc0 11000000
byte 6: 0x45 01000101
byte 7: 0x4a 01001010
byte 8: 0xb 00001011
byte 9: 0x3 00000011
byte 10: 0x19 00011001
byte 11: 0x78 01111000
byte 12: 0x68 01101000
byte 13: 0xdb 11011011
byte 14: 0x4c 01001100
byte 15: 0x60 01100000
byte 16: 0x52 01010010
byte 17: 0x0 00000000
byte 18: 0x41 01000001
byte 19: 0xf2 11110010
byte 20: 0xea 11101010
byte 21: 0x70 01110000
byte 22: 0x9 00001001
byte 23: 0x88 10001000
byte 24: 0xac 10101100
Locktime: 00000000
```

6.3 Block解析结果

程序自动获取最新区块并调用 `parse_block()` 解析，输出结果如下：

字段	值
Block Height	5075729
Version	623501312
Previous Block Hash	0000000000002109d7ac16eedb57b0563b879d8ddfc9e2181f82d5eccc8e33dc6
Merkle Root	cb87e51f613b61abc140b1ecabfb35bf6ed1769b20f4ae2cbf16bd48e601204b
Timestamp	1784561367
Nonce	3837886570
Difficulty	10659.334663609017

链上验证： 交易所在区块实际高度为 5075729，区块链浏览器显示交易已获得 2 个确认，说明交易有效且已被网络接受。

【4】解析区块数据

请输入区块哈希（或直接按Enter自动获取）：

自动获取最新区块：00000000000274d76c69543eee7d0e08717283ed572ebd497edee27978e97058

===== BLOCK =====

Height: 5075729

Version: 623501312

Previous hash: 000000000002109d7ac16eedb57b0563b879d8ddfc9e2181f82d5eecc8e33dc6

Merkle Root: cb87e51f613b61abc140blecabfb35bf6ed1769b20f4ae2cbf16bd48e601204b

Timestamp: 1784561367

Nonce: 3837886570

Difficulty: 10659.334663609017

=====

☒ 全部完成!

>>>

Ln: 42 Col: 0

七、结果分析

7.1 Bitcoin交易结构分析

7.1 Bitcoin 交易结构分析

本次解析的交易包含 1 个输入 和 2 个输出，结构清晰体现了比特币的 UTXO 模型。

1.输入分析：

该交易引用了上一笔交易（4ae4af4c...）的 Index 1 输出作为资金来源，金额为 270110 Satoshi。输入中包含解锁脚本（ScriptSig），由 71 字节的 DER 编码 ECDSA 签名和 33 字节的公钥组成。

2.输出分析：

交易产生两个新输出：

Output 0: 70110 Satoshi（转账给接收方 mmVL6H...）

Output 1: 199096 Satoshi（找零返回发送方 mxHfYit...）

3. 手续费计算：

手续费 = 输入金额 - 输出金额 = 270110 - 70110 - 199096 = 904 Satoshi，与链上显示的 0.00000904 tBTC 完全一致。

7.2 Bitcoin Script分析

Unlocking Script（解锁脚本）：

逐字节解析：

- 0x47：后续 71 字节

- 0x30 0x44：DER 编码开始，68 字节签名数据
- 0x02 0x20：R 值（32 字节）
- 0x02 0x20：S 值（32 字节）
- 0x21：后续 33 字节
- 0x02：压缩公钥标记
- 后 32 字节：公钥 X 坐标

验证流程：

1. 签名和公钥被压入栈
2. 锁定脚本执行：OP_DUP + OP_HASH160 + OP_EQUALVERIFY + OP_CHECKSIG
3. 用公钥验证签名，验证通过则允许花费 UTXO

Locking Script（锁定脚本）：

Output 0 的锁定脚本：`76a91441821e33c05a8de315e7c08d463dbc5f30b0036588ac`

这是标准 P2PKH脚本：

操作码	十六进制	说明
OP_DUP	0x76	复制栈顶公钥
OP_HASH160	0xa9	计算公钥哈希
OP_PUSH20	0x14	压入 20 字节公钥哈希
（数据）	41821e33...0b0036	20 字节公钥哈希
OP_EQUALVERIFY	0x88	比较是否匹配
OP_CHECKSIG	0xac	验证签名

Output 0
Value: 70110
Locking Script: 76a91441821e33c05a8de315e7c08d463dbc5f30b0036588ac

```
===== BYTE / BIT =====
byte 0: 0x76 01110110
byte 1: 0xa9 10101001
byte 2: 0x14 00010100
byte 3: 0x41 01000001
byte 4: 0x82 10000010
byte 5: 0x1e 00011110
byte 6: 0x33 00110011
byte 7: 0xc0 11000000
byte 8: 0x5a 01011010
byte 9: 0x8d 10001101
byte 10: 0xe3 11100011
byte 11: 0x15 00010101
byte 12: 0xe7 11100111
byte 13: 0xc0 11000000
byte 14: 0x8d 10001101
byte 15: 0x46 01000110
byte 16: 0x3d 00111101
byte 17: 0xbc 10111100
byte 18: 0x5f 01011111
byte 19: 0x30 00110000
byte 20: 0xb0 10110000
byte 21: 0x3 00000011
byte 22: 0x65 01100101
byte 23: 0x88 10001000
byte 24: 0xac 10101100
```

7.3 UTXO模型分析

本交易的 UTXO 流转过程：



UTXO 模型特点验证：

- 1. 每个交易输入引用一个已有的 UTXO，并将其销毁
- 2. 每个交易输出创建一个新的 UTXO，等待被未来交易引用
- 3. 不存在“账户余额”概念，只存在未花费交易输出集合
- 4. 验证交易只需检查输入是否存在于 UTXO 集以及签名是否有效

7.4 Block Header分析

字段	作用	本次值
Version	区块格式版本号	623501312
Previous Block Hash	指向前一个区块的哈希指针	000000000002109d...
Merkle Root	所有交易的 Merkle 树根哈希	cb87e51f...
Timestamp	区块时间戳（Unix 秒）	1784561367
Difficulty	当前挖矿难度目标	10659.33
Nonce	随机数（PoW 搜索变量）	3837886570

区块链不可篡改性分析：

本区块的 Previous Block Hash 指向前一个区块。若篡改本区块任何数据，区块哈希会发生变，后续区块的 Previous Hash 仍指向旧哈希，导致链断裂。

攻击者需重算所有后续区块的 PoW，计算成本极高。

PoW 机制分析：

Difficulty = 10659.33，表示当前挖矿难度，矿工通过不断调整 Nonce 值，使区块头双 SHA-256 哈希值小于目标难度；Nonce = 3837886570 是矿工找到的有效随机数。

八、实验总结

本实验基于 Bitcoin Testnet 网络，完成了交易数据和区块数据的获取与解析。通过编写 Python 程序，实现了对 Transaction 和 Block 中各字段的解析，并对 Script 进行了字节级分析，加深了对 Bitcoin 数据结构和交易流程的理解。

实验过程中，进一步掌握了 UTXO 模型、Bitcoin Script 以及 Block Header 的组成，理解了区块之间通过哈希值连接形成区块链，以及 PoW 共识机制在保障区块链安全性中的作用。

总体而言，本实验完成了预期目标，提高了对 Bitcoin 底层协议和数据结构的理解。后续可进一步完善程序，实现 SegWit 交易解析、Merkle Root 验证以及 Bitcoin Script 执行模拟等功能，以更深入地学习 Bitcoin 协议的实现原理。

作业二：libsecp256k1 中密码算法的漏洞修复与性能优化分析

ECDSA – Forge signature when the signed message is not checked

- Key Gen: $P = dG$, n is order
- Sign(m)
 - $k \leftarrow \mathbb{Z}_n^*$, $R = kG$
 - $r = R_x \bmod n, r \neq 0$
 - $e = \text{hash}(m)$
 - $s = k^{-1}(e + dr) \bmod n$
 - Signature is (r, s)
- Verify (r, s) of m with P
 - $e = \text{hash}(m)$
 - $w = s^{-1} \bmod n$
 - $(r', s') = e \cdot wG + r \cdot wP$
 - Check if $r' == r$
 - Holds for correct sig since
 - $es^{-1}G + rs^{-1}P = s^{-1}(eG + rP) =$
 - $k(e + dr)^{-1}(e + dr)G = kG = R$
- $\sigma = (r, s)$ is valid signature of m with secret key d
- If only the hash of the signed message is required
- Then anyone can forge signature $\sigma' = (r', s')$ for d
- (Anyone can pretend to be someone else)
- Ecdsa verification is to verify:
 - $s^{-1}(eG + rP) = (x', y') = R'$, $r' = x' \bmod n == r$?
 - To forge, choose $u, v \in \mathbb{F}_n^*$
 - Compute $R' = (x', y') = uG + vP$
 - Choose $r' = x' \bmod n$, to pass verification, we need
 - $s'^{-1}(e'G + r'P) = uG + vP$
 - $s'^{-1}e' = u \bmod n \rightarrow e' = r'uv^{-1} \bmod n$
 - $s'^{-1}r' = v \bmod n \rightarrow s' = r'v^{-1} \bmod n$
 - $\sigma' = (r', s')$ is a valid signature of e' with secret key d

*Project: check repo <https://github.com/bitcoin-core/secp256k1>, write a report on the bug fix, performance improvement related to crypto algo usage in bitcoin, figure out the why and the math behind

一. 实验背景

随着互联网应用的发展，越来越多的信息需要通过开放网络进行传输，例如：

- 金融交易；
- 区块链交易；
- 软件发布；
- 身份认证；
- 电子合同。

由于网络环境是不可信的，攻击者可能：

- 修改传输的数据；
- 冒充合法用户发送信息；
- 重放历史数据。

因此，需要一种技术能够同时保证：

1. 消息来源可信；
2. 消息内容未被修改；
3. 发送者无法否认自己的行为。

数字签名是 Bitcoin 中保证交易真实性的重要机制，而 Bitcoin Core 对其进行了工业级优化，因此有必要分析其密码库 libsecp256k1 的设计与实现。

二、实验目的

1. 熟悉 Bitcoin Core 开源密码库 **libsecp256k1** 的项目结构及主要功能。
2. 阅读 GitHub 仓库中的 CHANGELOG、Pull Request、Commit 及相关源码，了解密码算法相关的漏洞修复和性能优化。
3. 分析典型案例的产生原因、修复方法及数学原理，加深对密码工程实现的理解。

三、实验环境

项目	内容
操作系统	Windows
实验算法	ECDSA
椭圆曲线	secp256k1
哈希算法	SHA-256
开源仓库	https://github.com/bitcoin-core/secp256k1

四、实验原理

4.1 ECDSA算法原理

椭圆曲线数字签名算法是一种基于椭圆曲线密码学的数字签名算法，具有密钥长度短、安全强度高和计算效率较高等特点，被广泛应用于区块链、数字证书、电子签名以及身份认证等领域。Bitcoin 正是采用 ECDSA 对交易进行签名，以保证交易来源真实、内容完整且不可否认。

ECDSA主要包括密钥生成、数字签名和签名验证三个过程。

4.1.1 ECDSA密钥生成过程

选择随机私钥： $d \in [1, n-1]$

- d ：私钥；
- n ：椭圆曲线阶。

计算公钥： $P=dG$

- G ：椭圆曲线基点；
- P ：公开密钥。

攻击者可以获得： G, P

但是无法根据： $P=dG$ ，计算私钥 d

4.1.2 ECDSA签名过程

对于消息 m ，首先计算消息摘要： $e = \text{Hash}(m)$

然后随机选择一次性随机数： k

计算： $R = kG$

令： $r = x_R \bmod n$

计算： $s = k^{-1}(e + dr) \bmod n$

最终生成数字签名： (r, s)

4.1.3 ECDSA验证过程

验证者获得：

- 消息 m
- 签名 (r, s)
- 公钥 P

首先计算： $e = \text{Hash}(m)$

然后： $w = s^{-1} \bmod n$

计算： $R = w(eG + rP)$

如果满足： $x_R \bmod n = r$

则认为签名有效。

4.2 Bitcoin中的ECDSA应用背景

Bitcoin采用secp256k1椭圆曲线实现交易签名。

用户拥有私钥 d ，通过椭圆曲线点乘计算公钥： $P = dG$

其中：

- d 为私钥；
- G 为基点；
- P 为公开密钥。

交易发送过程中，用户使用私钥对交易数据Hash进行签名： (r, s)

网络节点通过公钥验证 $\text{Verify}(P, r, s, H(m))$

只有验证成功后，交易才会被加入区块。

理论上，只要 ECDSA 数学算法正确即可完成签名和验证；但在实际工程中，仅有数学正确性还远远不够，还需要兼顾运行效率、安全性以及跨平台一致性等因素。因此，Bitcoin Core 并未直接调用普通 ECDSA 实现，而是开发并维护了专用密码库 libsecp256k1，负责完成 Bitcoin 中所有与椭圆曲线密码相关的核心计算。

4.3 为什么Bitcoin需要专用密码库

原始ECDSA算法虽然数学安全，但直接实现存在两个问题：

第一：效率不足。

ECDSA大量时间消耗于 kG ，即椭圆曲线标量乘法。

例如签名 $R=kG$ ，验证 $R=u_1G+u_2PR=u_1G+u_2PR=u_1G+u_2P$

其中均包含大整数点乘。

第二：安全风险。

普通实现可能存在：

- 时间侧信道；
- 随机数泄露；
- 输入异常。

因此Bitcoin Core没有直接使用普通ECDSA库，而开发了专用库：

libsecp256k1。

4.4 椭圆曲线点乘优化

（1）普通实现

传统方法， $kG=G+G+G+...+G$ ，需要大量点加操作。

（2）窗口预计算

libsecp256k1采用预计算方式。提前计算 $G, 2G, 3G, ...$ 运行时根据 k 的二进制窗口直接组合。

例如：

普通： $13G=G+G+...$ ， $13G=G+G+...$ ， $13G=G+G+...$

优化： $13G=8G+4G+G$ ， $13G=8G+4G+G$ ， $13G=8G+4G+G$

减少计算次数。

4.5 Jacobian坐标优化

传统仿射坐标： (x,y)

点加需要：

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}$$

其中包含模逆运算，计算复杂。因此libsecp256k1采用Jacobian坐标 (X,Y,Z) ，通过坐标转换避免频繁求逆，最终提高大量点运算效率。

4.6 Constant-time安全优化分析

除了提高计算效率之外，Bitcoin密码库还重点考虑实现安全性。

在普通ECDSA实现中，如果根据秘密参数执行不同数量的操作，例如：

```
if k_bit == 1:
    point_add()
```

则程序运行时间可能随着私钥或随机数k的不同而变化。

攻击者通过大量测量签名时间，可能推测私钥相关信息，这类攻击称为：

Timing Side Channel Attack（时间侧信道攻击）

libsecp256k1采用constant-time设计，使关键密码运算过程：

- 不依赖秘密数据分支；
- 不产生明显时间差异；
- 避免缓存访问模式泄露。

因此，即使攻击者能够多次调用签名接口，也难以通过运行时间恢复私钥。

4.7 ECDSA随机数优化

ECDSA签名： $s=k^{-1}(e+dr)\pmod n$ ，其中k是一次性随机数。

如果两个不同消息使用相同k，则： $s_1=k^{-1}(e_1+dr)$ ， $s_2=k^{-1}(e_2+dr)$ ，攻击者可以计算d，因此随机数k的安全性与私钥安全性等价。

Bitcoin实际实现中采用RFC6979确定性随机数生成方式，即： $k=f(\text{private key,message})$ ，使同一私钥，同一消息，产生确定但不可预测的k。避免随机数质量不足导致私钥泄露。

五、源码分析和典型案例

5.1 libsecp256k1 项目结构分析

首先从 Bitcoin Core 官方 GitHub 仓库获取项目源码，下载源码后，对项目整体结构进行了阅读，并重点分析了 README.md、CHANGELOG.md、Release Note、Pull Request 以及部分核心源码文件。主要目录结构如下：

目录	作用
include/	对外提供的 API 接口
src/	密码算法核心实现，包括 ECDSA、ECDH、ECMult、Field、Scalar 等模块
examples/	官方示例程序

tests/	单元测试代码
bench/	性能测试程序
CHANGELOG.md	版本更新记录
README.md	项目整体说明

其中，本实验重点分析了 `src/` 目录中与 ECDSA、ECDH 及椭圆曲线点乘相关的源码，并结合 Release Note、Pull Request 与 Commit，对典型漏洞修复和性能优化进行了研究。

https://github.com/bitcoin-core/secp256k1

secp256k1Public

Watch135Fork1.2kStar2.5k

master2 Branches11 Tags

Go to fileTAdd fileCode

real-or-random Merge #1765: Add "silentpayments" module implementing BIP35... d2d0486 · 2 days ago 2,822 Commits

.github	ci: enable silentpayments module	2 weeks ago
autotools-aux/m4	autotools: Rename build-aux to autotools-aux	8 months ago
ci	ci: enable silentpayments module	2 weeks ago
cmake	cmake: Fix shared library versioning on OpenBSD	last month
contrib	include: in doc, remove article in front of "pointer"	3 years ago
doc	release process: mention the [Unreleased] link clearly	6 months ago
examples	silentpayments: add examples/silentpayments.c	2 weeks ago
include	silentpayments: drop "shuffle outputs" recommendation fro...	5 days ago
sage	sage: verify Eisenstein integer connection for GLV constants	6 months ago
src	Merge #1765: Add "silentpayments" module implementing Bl...	2 days ago
tools	tests: add BIP-352 test vectors	2 weeks ago
.gitattributes	Split off .c file from precomputed_ecmult.h	5 years ago
.gitignore	silentpayments: add examples/silentpayments.c	2 weeks ago

About

Optimized C library for EC operations on curve secp256k1

ReadmeMIT licenseContributingSecurity policyActivityCustom properties2.5k stars135 watching1.2k forksReport repository

Releases11

libsecp256k1 0.7.1Latest6 months ago+ 10 releases

Packages

No packages published

5.2 Bug Fix 分析

5.2.1 Constant-Time 漏洞修复

(1) 问题背景

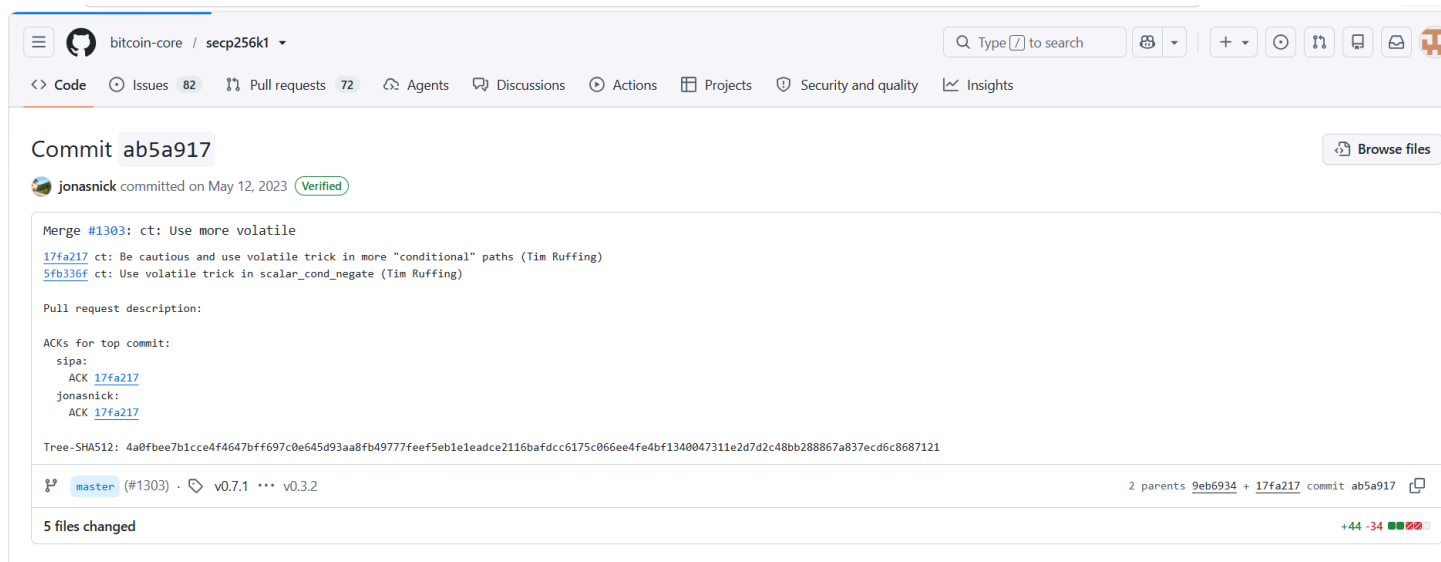
在 libsecp256k1 v0.3.2 Release Note 中，官方指出 GCC 13.1 编译器可能破坏 ECDH 模块的 Constant-Time 特性，从而导致时间侧信道攻击。ECDH 的核心计算如下：

$$P = dQ$$

其中：

- d ：私钥（Private Key）
- Q ：公钥（Public Key）

其本质仍属于椭圆曲线标量乘法，因此需要保证整个计算过程不会因秘密数据不同而产生不同的执行时间。



(2) 问题原因

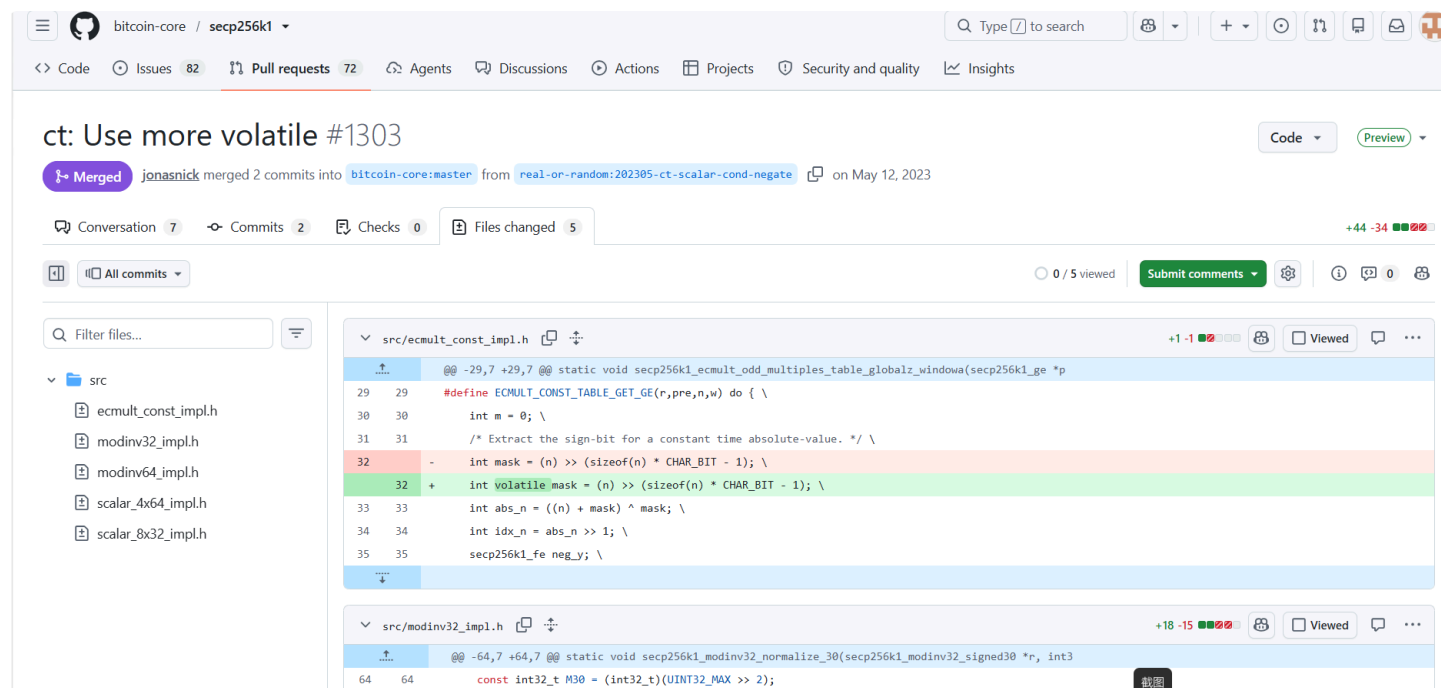
密码算法不仅要求数学正确，还要求程序执行时间不能依赖秘密数据。在 GCC 13.1 中，编译器为了提高程序性能，对部分依赖秘密数据的布尔变量进行了优化，使程序执行路径可能随着秘密数据发生变化，从而破坏 Constant-Time 特性。

该问题并非 ECDH 算法本身存在错误，而是编译器优化导致的软件实现漏洞。

(3) 修复方法

官方在多个源码文件中增加了 `volatile` 修饰，并修改了部分 `flag`、`mask` 和 `cond` 等变量的实现方式，阻止 GCC 对关键变量进行可能影响 Constant-Time 的优化。

修复后的程序虽然没有改变密码算法本身，但保证了程序执行路径不会因为秘密数据不同而发生变化，从而避免了时间侧信道攻击。



(4) 原因分析

Constant-Time 的核心思想是使程序运行过程与秘密数据无关。如果程序根据私钥不同执行不同数量的指令，即使数学计算完全正确，攻击者仍可能通过统计运行时间恢复私钥信息。因此，密码工程不仅需要保证算法正确，还必须保证实现过程具有 Constant-Time 特性。

(5) 数学原理

漏洞修复前后，ECDH 的数学过程始终保持不变：

$$P = dQ$$

因此，本次修复并未改变密码算法，而是修复了软件实现层面的安全问题，体现了密码工程中"数学正确"与"实现安全"同样重要。

5.2.2 Sensitive Data Clearing 安全机制分析

(1) 问题背景

密码算法运行过程中，私钥、随机数以及共享密钥等敏感数据会暂时保存在内存中。如果程序结束后没有及时清除，这些数据仍可能保留在内存中，存在密钥泄露风险。

(2) 问题原因

通常可以使用以下代码：

代码块

```
1  memset(buffer, 0, size);
```

清除内存。

然而现代编译器可能认为该内存不会再次访问，因此直接将这条语句优化掉（Dead Store Elimination），导致源码中虽然进行了清零，但实际生成的机器代码并未真正执行内存清除。

(3) 修复方法

libsecp256k1 没有直接使用普通 `memset()`，而是采用专门的安全内存清除方法，使编译器无法删除清零操作，从而确保私钥和随机数能够真正从内存中擦除。

(4) 原因分析

敏感数据在内存中的生命周期越长，被调试器、崩溃转储或恶意程序获取的风险越高。因此，在密码软件中及时、安全地擦除内存是保障密钥安全的重要措施。

(5) 数学原理

该修复并未改变任何密码算法公式，而是增强了密码实现过程中的安全性，体现了密码工程对于密钥生命周期管理的要求。

5.3 Performance Improvement 分析

5.3.1 Point Multiplication Optimization

(1) 性能瓶颈

椭圆曲线点乘是整个密码库中最耗时的运算。

ECDSA 签名需要计算： $R = kG$

ECDSA 验签需要计算： $R = u_1G + u_2P$

ECDH 密钥交换需要计算： $P = dQ$

因此，点乘运算直接决定整个密码库的运行效率。

(2) Window Method

传统 Double-and-Add 算法按照标量二进制逐位进行运算，需要大量倍点和点加操作。

libsecp256k1 采用 Window Method，将多个二进制位组成一个窗口进行统一处理，并结合预计算结果完成点乘。

例如：

普通计算 $13G = G + G + \dots + G$

窗口法计算： $13G = 8G + 4G + G$

虽然数学结果完全一致，但减少了点加次数，提高了整体计算效率。

(3) 数学原理

窗口法本质上属于计算顺序优化，并未改变标量乘法 kG 的数学定义，而是利用预计算结果减少运行时的椭圆曲线运算次数，因此能够在保持正确性的同时显著提升性能。

5.3.2 Jacobian Coordinate Optimization

传统仿射坐标采用 (x, y) 表示椭圆曲线点，每次点加和倍点运算都需要进行有限域模逆运算，而模逆是整个有限域计算中最耗时的操作之一。

libsecp256k1 使用 Jacobian 坐标

$$(X, Y, Z)$$

代替传统仿射坐标，通过坐标转换将大量模逆运算替换为乘法运算，仅在最终输出结果时进行一次坐标转换，因此显著提高了点乘效率。

5.3.3 Precomputed Table Optimization

由于生成元 G 固定不变，因此可以提前计算多个倍点，例如 $G, 2G, 3G, \dots$ ，运行过程中直接查表即可，无需重复计算这些中间结果。

官方后续版本进一步扩大了预计算表规模，由约 22 KB 增加到 86 KB，虽然增加了一定的内存占用，但能够减少运行时的点加次数，提高整体执行效率。这属于典型的**时间换空间**优化策略。

六、实验总结

本实验围绕 Bitcoin Core 开源密码库 libsecp256k1 展开，通过阅读 GitHub 仓库中的源码、Release Note、CHANGELOG、Pull Request 以及 Commit，分析了密码库中的典型漏洞修复与性能优化方案。

在安全方面，重点分析了 Constant-Time 漏洞修复 和 Sensitive Data Clearing 两个典型案例。前者通过限制编译器优化保证程序执行路径与秘密数据无关，从而抵御时间侧信道攻击；后者采用安全内存清除机制，确保私钥、随机数等敏感数据能够真正从内存中擦除，降低密钥泄露风险。

在性能方面，分析了 Window Method、Jacobian 坐标 和 预计算表 等关键优化技术。这些方法均未改变 ECDSA 的数学原理，而是在保持算法正确性的基础上减少椭圆曲线运算次数，提高了数字签名和密钥交换的执行效率。

通过本次实验可以认识到，现代密码软件的安全不仅依赖于密码学理论，还依赖于高质量的软件工程实现。优秀的密码库需要在正确性、安全性、性能和可维护性之间取得平衡，而这正是 libsecp256k1 作为 Bitcoin Core 核心密码库的重要价值所在。